

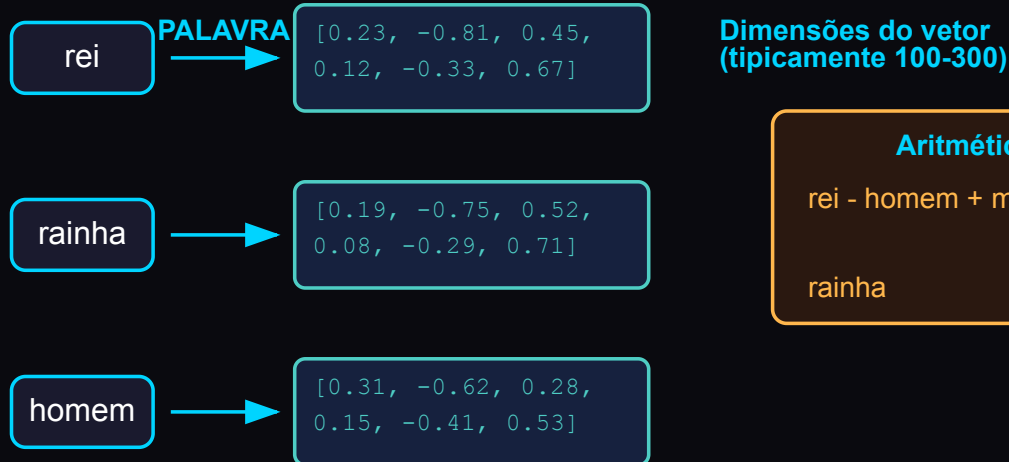
01

O que são Vetores (Embeddings)?

Como palavras são transformadas em vetores numéricos no espaço semântico

Espaço Semântico Vetorial

Como palavras se tornam vetores



Aritmética Vetorial

rei - homem + mulher

$\approx$

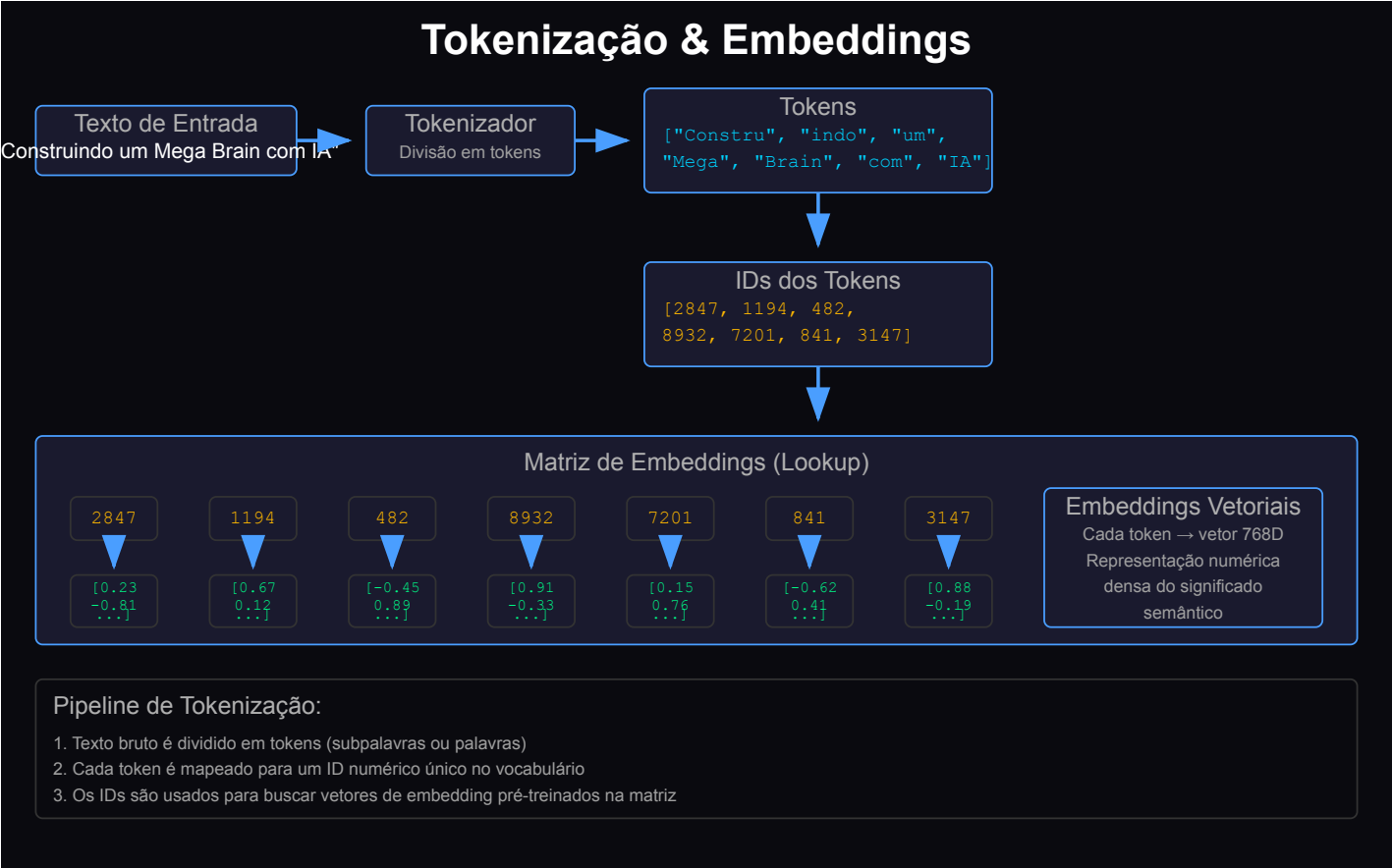
rainha

Cada palavra é convertida em um vetor numérico de alta dimensionalidade que captura seu significado semântico no espaço vetorial.

02

Tokenização: Texto → Tokens → Vetores

Pipeline completa: texto é quebrado em tokens, mapeado para IDs e convertido em embeddings



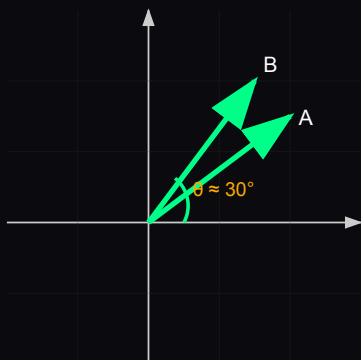
03

Similaridade de Cosseno entre Vetores

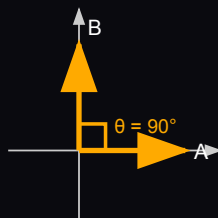
Como o LLM mede a proximidade semântica entre palavras usando o ângulo entre vetores

Similaridade de Cosseno em LLMs

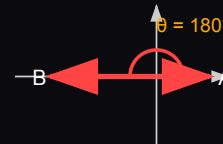
$$\cos(\theta) = (\mathbf{A} \cdot \mathbf{B}) / (|\mathbf{A}| \cdot |\mathbf{B}|)$$



Vetores Similares
 $\cos(\theta) \approx 0.87$
"gato" ↔ "felino"



Vetores Não Relacionados
 $\cos(\theta) = 0$
"gato" ↔ "matemática"



Vetores Opostos
 $\cos(\theta) = -1$
"bom" ↔ "ruim"

Interpretação: $\cos(\theta) \in [-1, 1]$

Próximo de 1: Muito similar

Próximo de 0: Não relacionado

Próximo de -1: Oposto

04

Mecanismo de Atenção (Self-Attention)

Cada token "olha" para todos os outros e decide quais são relevantes para o contexto

Self-Attention: cada token aprende a focar nos tokens relevantes

O gato bebeu leite porque estava com sede

Cada palavra "olha" para todas as outras e decide quais são relevantes

Matriz de Atenção (simplificada)

	O	gato	bebeu	leite	porque	estava	com	sede
gato								

Intensidade da Atenção:

Baixa Média Alta Muito Alta

Exemplo: "gato" presta atenção a:

- "bebeu" (ação do gato) - Atenção ALTA
- "sede" (motivo da ação) - Atenção ALTA
- "leite" (objeto da ação) - Atenção MÉDIA
- Outras palavras - Atenção BAIXA

O mecanismo de atenção permite que cada palavra identifique automaticamente quais outras palavras são mais relevantes para seu contexto

05

RAG — Retrieval Augmented Generation

Como combinar busca em documentos com LLMs para respostas precisas e fundamentadas

This page contains the following

error on line 65 at column 51: xml

Below is a rendering of the page

06

Ciclo do Agente de IA

O loop Perceber → Decidir → Agir → Observar que todo agente executa

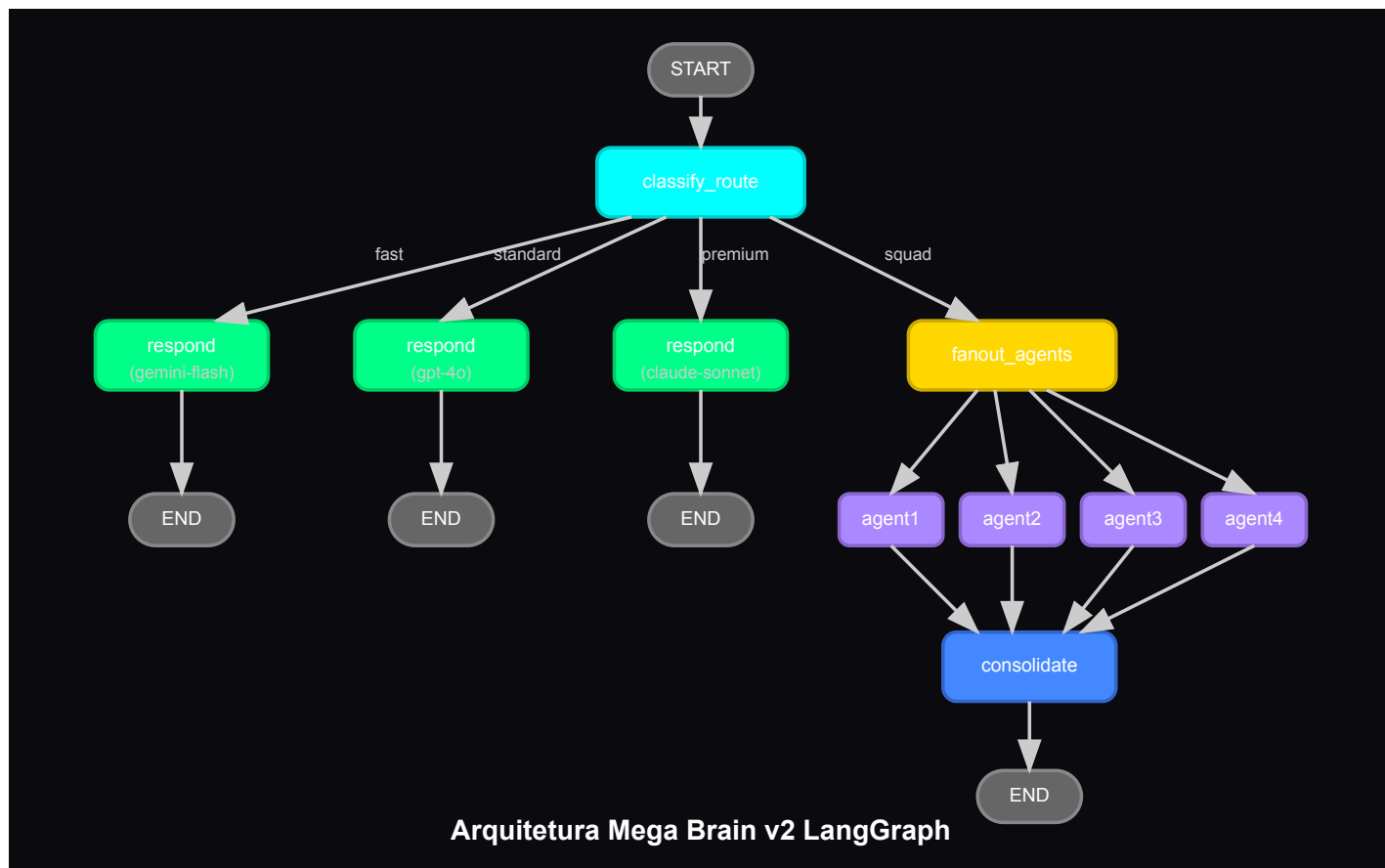


*Repete até encontrar
resposta satisfatória*

07

Arquitetura LangGraph — Mega Brain v2

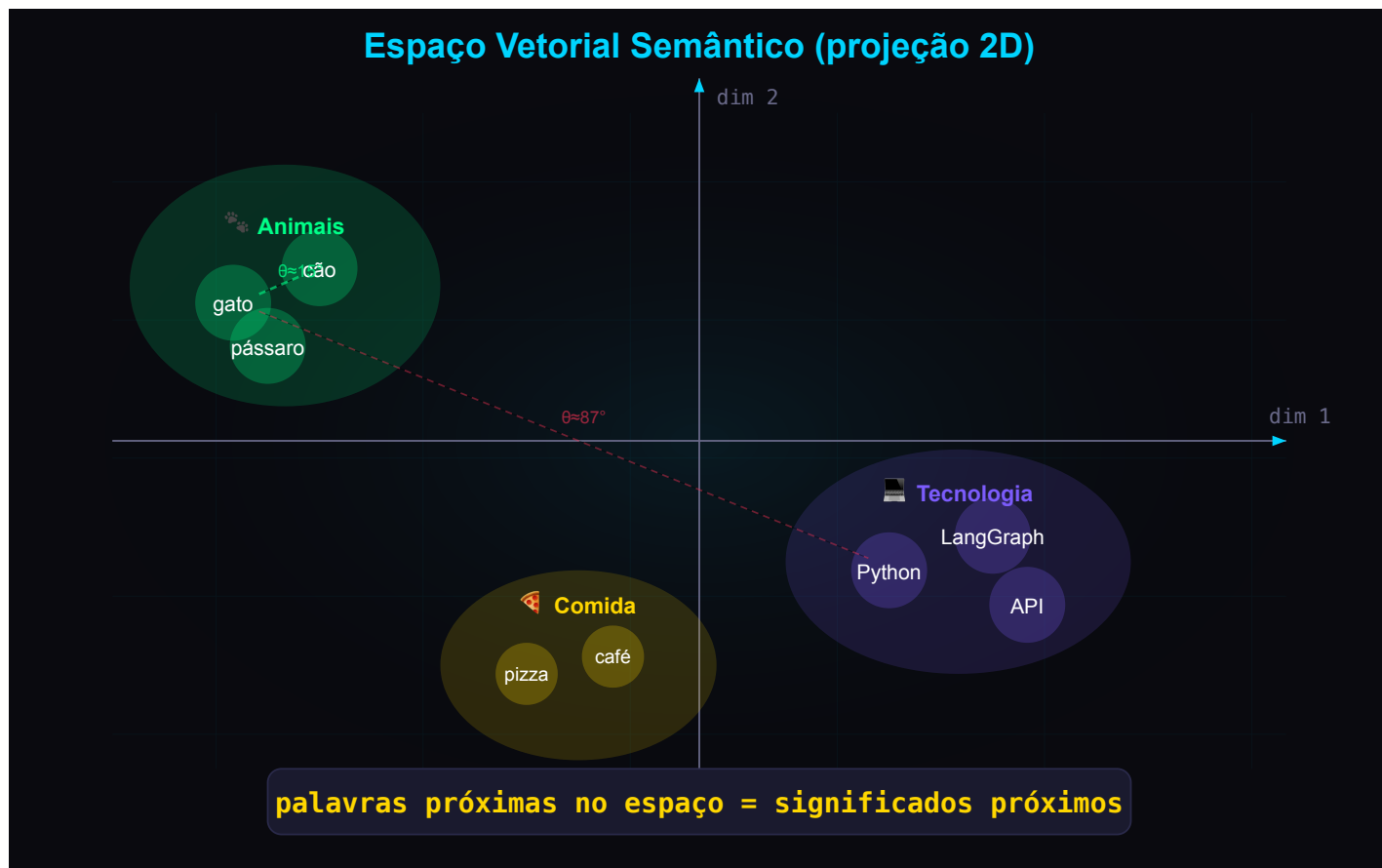
O grafo completo do Mega Brain v2: classificação, roteamento, squad e consolidação



08

Espaço Vetorial Semântico (Projeção 2D)

Visualização de como palavras com significados similares ficam próximas no espaço vetorial

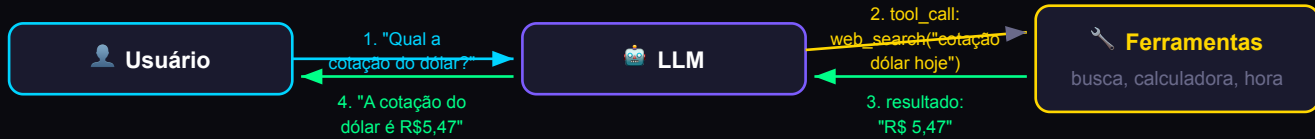


09

Tool Calling — Como o Agente usa Ferramentas

Fluxo completo do function calling: LLM decide qual ferramenta usar, código executa, LLM responde

Tool Calling — Como o Agente usa Ferramentas



Detalhamento do Fluxo (visão do código)

```
async def call_with_tools(messages, tools):
    # 1. Envia mensagem para o LLM com tools
    response = await openrouter.chat(messages, tools)

    # 2. Se o LLM decidiu usar uma ferramenta...
    if response.tool_calls:
        # 3. Executa a ferramenta localmente
        result = await execute_tool(tool_call.name, tool_call.args)

        # 4. Reenvia resultado para o LLM gerar resposta final
        return await call_with_tools(messages + [response, tool_result], tools)

    return response.content # Resposta direta
```

⚡ Importante

O LLM não executa a ferramenta. Ele só diz QUAL usar.

10

Execução Paralela com Send API

Comparação v1 (asyncio.gather manual) vs v2 (Send API declarativa) — fan-out e consolidação

Execução Paralela com Send API (LangGraph)

❌ v1: asyncio.gather (manual)

```
agent1 = call("copywriter", task)
agent2 = call("strategist", task)
agent3 = call("social", task)

results = await asyncio.gather(
    agent1, agent2, agent3
)

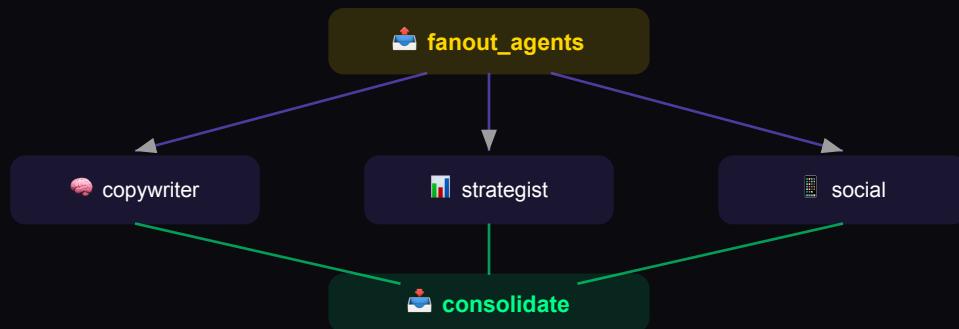
# + código manual de consolidação
```

✅ v2: Send API (declarativo)

```
def fanout (state):
    return [
        Send("agent", {
            "name" : n})
        for n in AGENTS
    ]

# LangGraph gerencia tudo
```

Fluxo do Squad Mode



⌚ Todos executam
ao mesmo tempo!